

JHQ-GPU: Representation-Driven GPU Execution and Workload-Adaptive Hierarchical Refinement

Rui Luo , Mengxuan Zhang*

Australian National University, Canberra, Australia
Rui.Luo1@anu.edu.au, mengxuan.zhang@anu.edu.au

Abstract. Reducing approximate nearest-neighbor (ANN) query time requires efficient screening and selective refinement. We present JHQ-GPU, which optimizes some stages of JHQ through representation-aware execution and workload-level budget calibration. Cartesian codebook factorization replaces each 256-entry distance table with two 16-entry tables, preserving the represented distance while reducing storage eight-fold. Candidate-parallel scanning and packed access complement this design. Refinement budgets are calibrated through sampled top- k agreement with a larger reference budget, without ground-truth labels or a learned predictor. Across six datasets, two-group factorization achieves the highest observed throughput in 23 of 24 configurations. Calibration delivers $1.05\text{--}2.77\times$ the mean held-out throughput of fixed $\alpha = 100$ in ten of twelve cases, with $5.2\text{--}30.8$ ms calibration cost. These gains involve workload-dependent recall trade-offs; the remaining cases improve mean recall at lower throughput.

Keywords: Approximate nearest-neighbor search · GPU · Quantization · Adaptive query processing

1 Introduction

Approximate nearest-neighbor (ANN) search reduces query execution time by avoiding exhaustive, full-precision distance evaluation. Graph indexes restrict candidate visits through proximity-based navigation, as in CAGRA [11]. Quantization instead compresses vectors and accelerates candidate scoring, often combined with inverted-file (IVF) routing. Product quantization (PQ) [7] estimates distances through subvector lookup tables, while RaBitQ [5] uses randomized quantization with theoretical error bounds. Both seek inexpensive distance evaluation while retaining sufficient information to recover accurate neighbors.

JHQ [6] separates these requirements through hierarchical quantization. Following an orthogonal transform, compact primary codes screen routed candidates, and residual codes refine selected survivors. This avoids detailed scoring across the entire candidate pool while retaining a path to more accurate ranking.

* Corresponding author.

The benefit is particularly relevant to high-dimensional vectors, where residual processing is expensive, but depends on efficient screening and selective refinement.

GPUs expose parallelism across candidates and queries. Faiss-GPU [8] provides optimized PQ search and top- k selection, and develops quantization-specific search kernels on GPU. These systems demonstrate that acceleration requires matching the representation to GPU execution. For JHQ, this raises two connected challenges: reducing primary scoring cost and controlling the refinement work that follows.

First, compact primary codes do not imply compact distance tables. A direct implementation uses 256 entries per subspace, potentially exceeding available shared memory. JHQ’s Cartesian codebook permits exact decomposition into two 16-entry tables without changing the represented distance. However, further decomposition increases lookup work.

Second, faster screening can leave refinement as the dominant cost. At $\alpha = 100$ and $nprobe = 32$, refinement accounts for 57% and 70% of timed work on Vogue and OpenAI-3072, respectively. A large fixed budget may refine candidates that do not affect the output, whereas a small budget may discard useful neighbors. We therefore calibrate the budget through sampled top- k agreement with a larger reference budget and reuse it across subsequent batches. This requires neither ground-truth labels nor a learned predictor, although agreement does not guarantee recall.

We implement these designs in JHQ-GPU and evaluate six datasets. Two-group factorization achieves the highest observed throughput in 23 of 24 configurations. With 128 calibration queries, selected budgets yield $1.05\text{--}2.77\times$ the mean held-out throughput of fixed $\alpha = 100$ in ten of twelve configurations; the remaining two improve mean recall at lower throughput. The evaluation separates scan acceleration from refinement-related quality trade-offs and calibration overhead.

Contributions

- **Representation-aware GPU execution.** We combine exact Cartesian distance-table factorization with candidate-parallel scanning and packed access, evaluating the trade-off between table footprint and lookup work.
- **Workload-adaptive refinement.** We calibrate refinement budgets using unlabeled query outputs, analyze the criterion’s monotonicity conditions and calibration amortization, and assess held-out quality.
- **Experimental evaluation.** Across six datasets, we evaluate execution choices and budget policies alongside IVF-PQ and CAGRA, quantifying performance, quality, construction, and memory trade-offs.

2 Preliminaries

Approximate Nearest-Neighbor Search Given a database $\mathcal{X} \subset \mathbb{R}^d$ of N vectors and a query $\mathbf{q}$, nearest-neighbor search returns the k vectors minimizing

$D(\mathbf{q}, \mathbf{x}) = \|\mathbf{q} - \mathbf{x}\|_2^2$. Approximate nearest-neighbor (ANN) search reduces execution cost by allowing deviations from the exact result. For approximate and exact top- k sets $\mathcal{R}(\mathbf{q})$ and $\mathcal{R}^*(\mathbf{q})$, quality is measured by

$$\text{Recall@}k(\mathbf{q}) = \frac{|\mathcal{R}(\mathbf{q}) \cap \mathcal{R}^*(\mathbf{q})|}{k}. \quad (1)$$

JHQ Representation JHQ [6] applies a square orthogonal transform $\mathbf{y} = \mathbf{Q}\mathbf{x}$ and $\mathbf{q}' = \mathbf{Q}\mathbf{q}$, where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. This preserves Euclidean distances without reducing dimensionality. The transformed vector is partitioned into M subspaces of dimension D_s , with $d = MD_s$. Each B -bit primary code $c_m(\mathbf{x})$ selects a codeword from $\mathcal{C}_m$, whose 2^B entries are Cartesian products of one-dimensional Lloyd–Max quantization levels.

Concatenating the selected codewords gives the primary reconstruction $\hat{\mathbf{y}}^P$. JHQ additionally quantizes the residual $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}^P$ using B_r bits per dimension, yielding $\hat{\mathbf{r}}$ and the hierarchical reconstruction $\hat{\mathbf{y}}^H = \hat{\mathbf{y}}^P + \hat{\mathbf{r}}$. We use $B = D_s = 8$ and $B_r = 8$, giving one primary bit and eight residual bits per dimension, excluding index metadata. The residual is defined relative to the primary reconstruction, not an IVF centroid.

Hierarchical Query Execution IVF routing selects inverted lists containing n_c candidates. For each subspace, a query-specific table stores $T_m[c] = \|\mathbf{q}'_m - \mathcal{C}_m[c]\|_2^2$. Primary screening scores each candidate through $D_P(\mathbf{q}, \mathbf{x}) = \sum_{m=1}^M T_m[c_m(\mathbf{x})]$. The query is not quantized into a database code. Screening retains up to $c_k = \max\{k, \lceil \alpha k \rceil\}$ survivors, where α controls refinement effort. Only survivors access residual codes and receive the hierarchical score $D_H(\mathbf{q}, \mathbf{x}) = \|\mathbf{q}' - (\hat{\mathbf{y}}^P + \hat{\mathbf{r}})\|_2^2$. The final result contains the top- k survivors under D_H ; refinement cannot recover neighbors excluded by routing or screening.

Once tables are constructed, primary scoring requires $O(n_c M)$ work and residual scoring requires $O(c_k d)$ work for c_k survivors, excluding routing and selection. These costs motivate our two optimization targets: efficient GPU primary scoring and workload-dependent refinement budgets.

3 GPU Execution for JHQ

We accelerate primary screening through candidate-parallel access and exact table factorization, preserving JHQ’s transform, codebooks, and hierarchy.

3.1 Candidate-Parallel Scanning

Warp lanes score different candidates at the same subspace. A subspace-major scan copy, $[M, N]$, places consecutive candidates’ codes contiguously for coalesced access. Packing four byte codes into a 32-bit word reduces loads, address calculations, and loop iterations, with register-based unpacking. These optimizations preserve logical code volume but leave $256M$ query-specific distance entries, potentially exceeding shared-memory capacity.

3.2 Exact Distance-Table Factorization

For $B = D_s = 8$, JHQ’s Cartesian codebook associates the high and low four bits with separate coordinate groups. Squared-distance additivity gives $T_m[c] = T_m^{\text{hi}}[\lfloor c/16 \rfloor] + T_m^{\text{lo}}[c \bmod 16]$, where each subtable stores distances to 16 group codewords. This reduces storage eightfold and construction arithmetic from $256D_s$ to $32(D_s/2)$ coordinate contributions, a $16\times$ reduction relative to independent full-table construction. Exactness holds in real arithmetic; floating-point association and ties may differ. Arbitrary learned PQ subcodebooks do not generally provide this separability.

3.3 Grouping and Refinement Trade-offs

Equal G -way grouping, $G \in \{1, 2, 4, 8\}$, requires $G2^{8/G}$ entries per subspace and G lookups per candidate–subspace pair. Both $G = 4$ and $G = 8$ store 16 entries but require different lookup work; smaller tables therefore need not produce faster scans.

Tables use shared memory when $40 \cdot \text{BLOCK} + 4MG2^{8/G} \leq S_{\text{opt}}$, where the first term is candidate scratch, table entries occupy four bytes, and S_{opt} is the configured per-block limit. Otherwise, tables use global memory. With $S_{\text{opt}} = 101,376$ bytes and block sizes 128–1024, split tables at $M = 96/384$ require 12/48 KiB and fit, whereas full tables require 96/384 KiB and do not. Caching and instruction costs still determine the realized benefit.

Only primary survivors access residual codes, preserving $O(n_c M)$ screening and $O(c_k d)$ refinement. Faster screening leaves refinement work unchanged, motivating workload-level budget selection.

4 Adaptive Hierarchical Refinement

We balance refinement work and output stability by calibrating α on sampled queries and reusing it across batches.

4.1 Output-Stability Criterion

For fixed index, routing, and k , let $\mathcal{R}_\alpha(\mathbf{q})$ and $\mathcal{R}_{\max}(\mathbf{q})$ be top- k ID sets at budgets α and $\alpha_{\max}$. Over S sampled queries $\mathcal{Q}_s$, define

$$D(\alpha) = \sum_{\mathbf{q} \in \mathcal{Q}_s} (k - |\mathcal{R}_\alpha(\mathbf{q}) \cap \mathcal{R}_{\max}(\mathbf{q})|). \quad (2)$$

Acceptance requires $D(\alpha) \leq \epsilon$, where ϵ counts allowed disagreeing slots and ignores result order. Fixed ϵ becomes stricter as S increases. This label-free criterion neither guarantees recall nor detects improvements beyond $\alpha_{\max}$; generalization is evaluated on held-out queries.

4.2 Budget Search and Its Conditions

For descending grid $A[0], \dots, A[L-1]$ with $A[0] = \alpha_{\max}$, compute the reference once and set $l = 0$, $h = L - 1$. While $l < h$, test $j = \lceil (l + h)/2 \rceil$: set $l = j$ if $D(A[j]) \leq \epsilon$, otherwise $h = j - 1$. Return $A[l]$, an observed acceptable point with the reference as fallback. This requires at most $\lceil \log_2 L \rceil$ additional tests.

Proposition 1. *With fixed routing, exact top- c_k primary selection, nondecreasing c_k , fixed refined scores, and deterministic total tie orders, $D(\alpha_1) \geq D(\alpha_2)$ for $\alpha_1 \leq \alpha_2 \leq \alpha_{\max}$.*

Proof. Primary survivor sets nest: $\mathcal{S}_{\alpha_1} \subseteq \mathcal{S}_{\alpha_2} \subseteq \mathcal{S}_{\alpha_{\max}}$. A reference top- k item present in $\mathcal{S}_{\alpha}$ remains in its refined top- k , since restricting the set cannot add preceding items. Thus $|\mathcal{R}_{\alpha} \cap \mathcal{R}_{\max}| = |\mathcal{S}_{\alpha} \cap \mathcal{R}_{\max}|$ cannot decrease with α , proving the claim.

Under these assumptions, bisection finds the smallest acceptable grid value. The production selector retains only K_LOCAL=4 candidates per thread before merging, so exact selection and monotonicity are not guaranteed. We therefore compare the rule with enumeration. Held-out experiments use:

$A = (200, 100, 64, 32, 16, 8, 4, 2)$ and $\epsilon = 1$.

4.3 Calibration Cost and Reuse

For calibration cost T_{cal} and equal-sized batch times T_{fixed} , T_{rule} , adaptive execution costs $T_{\text{cal}} + BT_{\text{rule}}$ over B batches, versus BT_{fixed} . When $T_{\text{rule}} < T_{\text{fixed}}$, break-even occurs at

$$B^* = \frac{T_{\text{cal}}}{T_{\text{fixed}} - T_{\text{rule}}}. \quad (3)$$

Otherwise, calibration never repays through throughput. Reuse assumes stable queries, index, routing, and k ; recalibration every H batches adds T_{cal}/H per batch. Drift detection is untested. Measured calibration includes reference search and agreement checks but excludes sample gathering.

5 Related Work

Quantisation and grouped ADC. PQ [7] decomposes vectors into subspaces for compressed storage and asymmetric distance computation. JHQ [6] supplies our transform, Cartesian primary codebook and residual hierarchy. PQ Fast Scan [1] and Quicker ADC [2] establish small-table execution, including 16-entry tables. We study grouping and packed access for JHQ's representation.

GPU quantised search. IVF-RaBitQ [14] is the closest execution design: a cheap filter feeding a refinement stage, grouped lookup tables and a GPU layout. Two-stage execution and grouped lookup are shared ideas; ours are their measured trade-off for JHQ’s Cartesian codebook and the calibration of its residual budget. We do not report it as a baseline: we could not configure the GPU implementation available to us so that its search reproduced the two-stage behaviour its paper describes, and we prefer no measurement to one of a configuration we cannot vouch for. CAGRA [11] and cuVS IVF-PQ [10] supply the GPU comparisons; GPU compressed-domain scanning and k -selection go back to Faiss-GPU [8].

Adaptive search. Faiss [4] sweeps search-time settings and keeps the Pareto-optimal ones; learned early termination [9] controls per-query search effort. Our workload policy uses unlabelled output agreement to select JHQ’s residual budget, with enumeration as an empirical reference. It calibrates a shared budget for subsequent batches under stationary queries; it neither predicts per-query difficulty nor guarantees a target ground-truth recall.

6 Evaluation

We evaluate two system questions: how efficiently JHQ’s primary representation can be executed on a GPU, and how refinement demand varies across workloads. We then measure recall-throughput operating regions, calibration and held-out risk, controlled execution ablations, CPU/GPU budget sensitivity, and construction and memory costs.

6.1 Setup and protocol

Measurements use an RTX 5090 (32 GB, 170 SMs), a Xeon Platinum 8470Q host (208 logical CPUs), CUDA 13.0, driver 595.71.05, and cuVS 26.08.01. Table 1 lists the six datasets and routing configurations. OpenAI uses DBpedia embeddings, BGE Italian Wikipedia, and Stella TREC24 BioGen. For these datasets, preprocessing removes non-finite vectors, normalises nonzero vectors, and holds out queries with seed 42; Stella queries are corpus samples. All systems use the same exported vectors and squared-L2 top-10 ground truth.

Table 1. Database shape and JHQ routing; n_q is the actual query count.

Dataset	N	d	n_q	$nlist$
vogue-768 [3]	932,328	768	1,000	4,096
arxiv-768 [15]	2,253,000	768	1,000	8,192
bge-m3 [17]	10,091,524	1,024	1,000	32,768
stella-trec24 [16]	17,776,615	1,024	999	32,768
openai3-1536 [12]	999,000	1,536	1,000	8,192
openai3-3072 [13]	999,000	3,072	1,000	8,192

Frontier timing covers host query input to host result output for the entire query set, averaging three warmed runs and excluding calibration. Three cold Vogue builds yield recall 0.9852, 0.9842, and 0.9849; cross-build differences below approximately 10^{-3} therefore require repeated evidence. Small timing differences are treated cautiously rather than as decisive improvements.

JHQ uses $M = d/8$, $B = B_r = 8$, 39 training points per centroid, eight coarse iterations, and 100,000 residual-training vectors. Frontiers sweep $nprobe \in \{8, 32, 128, 256, 512, 1024\}$. The primary system comparison fixes $\alpha = 100$, separating execution performance from budget selection. The contextual calibrated curve uses $S = 32$, $\epsilon = 1$, and $\alpha_{\max} = 100$ on the evaluated pool. In a disjoint-half repetition, 18 of 36 selected budgets change although the mean selected-to-reference QPS ratio remains similar, 1.60 versus 1.59. Similar aggregate speed ratios can therefore hide different selected budgets, so calibration is evaluated separately on held-out queries.

IVF-PQ sweeps eight-bit subcodes at 96–768-byte code lengths; CAGRA sweeps graph degree, search width, and intermediate top- k . GPU memory is not matched. Frontiers retain nondominated valid points, and matched-recall ratios interpolate log QPS without extrapolation. Code and evaluation scripts will be released upon publication.

6.2 RQ1: End-to-end operating regions

Figure 1 shows workload-dependent operating regions rather than uniform dominance. CAGRA is $1.45\text{--}2.03\times$ faster on the 768-dimensional datasets at recall 0.90/0.95. JHQ approaches parity on OpenAI-1536 at 0.95 and reaches $1.52\times$ CAGRA’s QPS at 0.98; the corresponding OpenAI-3072 ratios are 0.66 and 1.09. IVF-PQ is slower at the evaluated targets within common measured coverage.

JHQ separates routing coverage from second-level work: increasing $nprobe$ visits more inverted lists, whereas αk bounds how many screened candidates receive residual scoring. Broader routing can therefore expand primary screening without requiring residual scoring of every additional candidate. At $\alpha = 100$ and $nprobe = 32$, residual refinement accounts for approximately 57% of measured work on Vogue and 70% on OpenAI-3072. Each retained candidate requires its residual code to be read and scored, so a large fixed survivor set remains costly even when primary screening is efficient. Reducing that set avoids residual work, but candidates discarded during screening cannot be recovered later. Refinement is therefore a substantial remaining cost in these measured cells, and the survivor budget controls both the available acceleration and the risk of losing useful neighbours.

For baseline fairness, IVF-PQ is reported under the better measured partitioning between its own and JHQ’s. CAGRA exceeds the device memory capacity during construction on BGE and Stella, so the missing points mark a feasibility boundary on this device rather than a search-quality failure. One contaminated Stella measurement is excluded throughout.

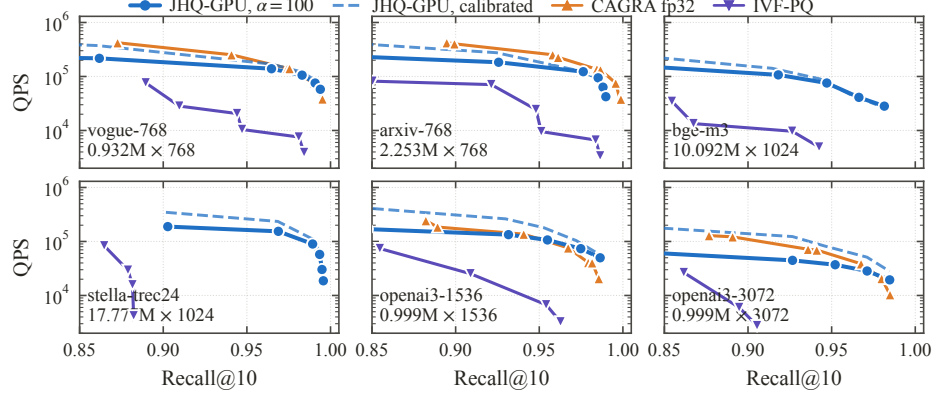


Fig. 1. Recall@10-QPS, entire query set per call. Solid JHQ fixes $\alpha = 100$; dashed calibrates on a 32-query sample of the evaluated pool and excludes calibration cost, so it is contextual rather than held out. Section 6.3 evaluates calibration on held-out queries. Endpoints are measured sweep endpoints.

6.3 RQ2: Budget selection, quality, and calibration cost

Protocol and workload variation. Each dataset uses one shared index for fixed budgets, sampled calibration, full-pool calibration, same-sample enumeration, and an offline oracle. Even-indexed queries calibrate; odd-indexed queries evaluate. The grid is $\{200, 100, 64, 32, 16, 8, 4, 2\}$ with $\epsilon = 1$. Across probes 128 and 512, the oracle chooses the fastest grid point within 10^{-3} of the best measured held-out recall. Its budgets range from 4 to 200, and the sampled rule’s median agrees in ten of twelve configurations at $S = 128$ (Table 2). This variation motivates workload-specific control of $c_k = \alpha k$: a shared constant can spend refinement work where little is gained or restrict it where additional candidates improve the result. Median agreement alone does not establish quality for every calibration draw.

Output stability and held-out quality. Figure 2(a) shows that 27–98% of draws at $S = 32$ lose more than 10^{-3} held-out recall relative to the best tested budget; at $S = 128$, counts range from zero to 40 of 64. For BGE at probe 512, 24 of 64 draws exceed the 10^{-3} threshold, while the mean held-out recall loss is approximately 0.0009 and the 95th percentile is 0.0014. The agreement test observes only sampled outputs: a budget sufficient for those queries may discard useful candidates for unseen queries, and the reference output itself is approximate. Output stability is therefore a label-free heuristic rather than a ground-truth recall guarantee. Increasing S also tightens the allowed disagreement fraction because $\epsilon = 1$ is fixed, so the observed change combines broader sampling with a stricter criterion; zero failures in repeated draws from fixed pools do not establish zero risk.

Table 2. Held-out budget comparison, $\epsilon = 1$, $\alpha_{\max} = 200$. Each cell gives α and Recall@10, then kQPS. Rule entries are the median α , mean recall and mean selected-arm QPS over 64 draws at $S = 128$.

Dataset / probe	Fixed 64	Fixed 100	Rule	Full pool	Oracle
vogue / 128	64; 0.9678	100; 0.9680	64; 0.9677	100; 0.9680	64; 0.9678
	143.2	118.7	143.4	118.7	143.2
	64; 0.9736	100; 0.9748	100; 0.9751	200; 0.9760	200; 0.9760
arxiv / 128	155.4	114.0	100.0	66.4	66.4
	64; 0.9280	100; 0.9280	8; 0.9279	8; 0.9280	8; 0.9280
	146.6	124.4	248.5	248.9	248.9
openai3-1.5k / 128	64; 0.9252	100; 0.9252	4; 0.9252	4; 0.9252	4; 0.9252
	81.5	42.4	117.5	117.5	117.5
	64; 0.9192	100; 0.9198	32; 0.9190	64; 0.9192	32; 0.9190
bge / 128	110.0	100.0	120.3	110.0	124.8
	64; 0.9892	100; 0.9892	16; 0.9884	16; 0.9886	16; 0.9886
	91.9	84.8	103.3	103.1	103.1
stella / 128	64; 0.9914	100; 0.9914	64; 0.9913	100; 0.9914	64; 0.9914
	72.4	64.8	71.1	64.8	72.4
	64; 0.9844	100; 0.9878	200; 0.9885	200; 0.9894	200; 0.9894
arxiv / 512	68.9	58.9	50.4	41.3	41.3
	64; 0.9740	100; 0.9740	8; 0.9738	8; 0.9740	8; 0.9740
	70.4	68.3	97.2	97.2	97.2
openai3-1.5k / 512	64; 0.9704	100; 0.9704	4; 0.9702	4; 0.9702	8; 0.9704
	39.0	27.5	49.4	49.4	49.6
	64; 0.9664	100; 0.9670	64; 0.9661	64; 0.9664	64; 0.9664
openai3-3k / 512	40.1	38.7	40.8	40.1	40.1
	64; 0.9954	100; 0.9954	16; 0.9945	16; 0.9948	16; 0.9948
	29.6	29.0	30.9	30.9	30.9
bge / 512					
stella / 512					

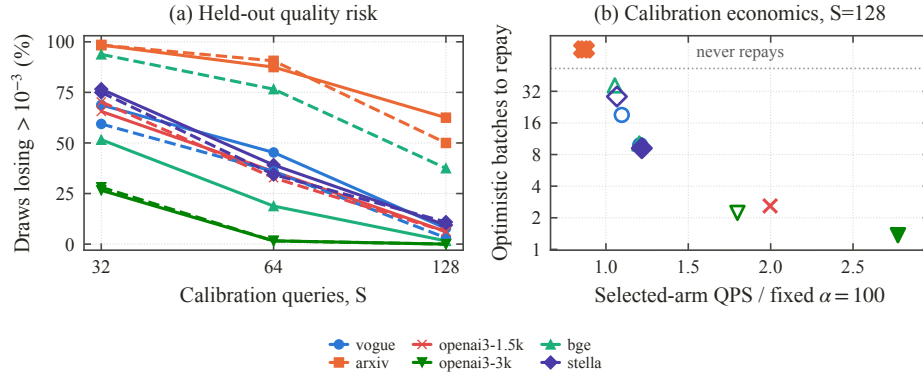


Fig. 2. (a) Draws losing $> 10^{-3}$ recall against the best tested budget; solid/dashed lines denote probes 128/512. (b) Mean selected-arm QPS gain and optimistic aggregate payback at $S = 128$, relative to fixed $\alpha = 100$; filled/open markers denote probes 128/512. Payback uses 500-query equivalents.

Table 3. Paired observations, each holding the represented distance fixed. *Cells* counts comparisons within a source experiment; the ranges do not describe a common workload.

Change	QPS effect	cells	Simple heuristic challenged
Table-free exact score	−30 to −52%	12	Minimum table state
$G = 8$ versus $G = 4$	−3.4 to −38.5%	24	Equal footprint, equal speed
Packed 32-bit loads	+1 to +66%	48	Bytes moved predict speed
Private probe cursor	−1.4 to +148.5%	20	Loop work is negligible

Searching the budget grid. Bisection and enumeration agree in all 2,304 draws across $S \in \{32, 64, 128\}$; enumeration costs 1.66–2.00 $\times$ as much, including reference search. Bisection saves work by testing only part of the grid, while the additional tests in enumeration do not change the selected budgets on these samples. Thus bisection makes the same sampled criterion cheaper; it does not remove the criterion’s held-out generalisation risk.

Benefit and reuse. At $S = 128$, mean selected-arm QPS is 1.05–2.77 $\times$ fixed- $\alpha = 100$ QPS in ten of twelve configurations. The arxiv configurations instead sacrifice 12–14% throughput for higher mean recall because matching a larger reference can require increasing the budget above the fixed comparison point. Calibration costs 5.2–30.8 ms, yielding an estimated payback of 1.4–36.4 batches of 500-query equivalents in configurations with throughput gains (Figure 2(b)). Calibration is paid once for a reusable budget, whereas savings accumulate over subsequent batches; short workloads may end before recovering this cost, and the slower arxiv selections never repay through throughput. Selected-arm QPS averages previously measured fixed-arm throughputs; taking the reciprocal of that average underestimates mean batch time when selections differ, so the aggregate payback is optimistic. Calibration includes reference search and agreement checks but excludes sample gathering, and reuse assumes a stable workload.

6.4 RQ3: Which execution choices account for the gains?

The hierarchy ablation changes representation fidelity and is discussed separately. The paired execution ablations in Table 3, by contrast, hold the represented distance fixed, allowing throughput differences to be attributed to execution choices rather than to a different scoring function.

Role of the hierarchy. Primary-only recall peaks at 0.5135–0.8642 across the six datasets, and residual refinement extends every range. Residual codes supply information omitted by the compact primary representation, improving the ranking among retained candidates. Applying that additional precision only after screening avoids residual scoring across the entire routed pool. Thus, α controls how many candidates receive residual scoring, while residual-code precision remains fixed. The ablation keeps the full allocation, so it does not measure the memory of a minimal primary-only index.

Table grouping. With represented distance fixed, $G = 2$ leads in 23 of 24 configurations (Figure 3); the exception favours $G = 1$ by only 0.03%. Splitting 256 entries into two 16-entry tables reduces table state eightfold with two lookups per candidate–subspace pair. Further splitting eventually stops saving table state: $G = 4$ and $G = 8$ both store 16 entries per subspace, but $G = 8$ requires twice as many lookups and is observed to be 3.4–38.5% slower. The extra lookups recur throughout the scan without reducing table footprint, showing why the smallest tables need not give the fastest search.

Packed access. Across 48 paired comparisons, packing improves QPS by 1–66%, with a maximum four-arm recall difference of 0.0001. A 32-bit load fetches four byte codes, amortising address calculation and loop control across them while adding register unpacking. The logical code volume and represented distance remain unchanged, so the improvement is consistent with more efficient code access rather than a different scoring approximation. Factorisation gains appear with both byte and word loads, supporting their combination. All arms use subspace-major storage, so transposition is not separately isolated; differences near 1% require stronger timing evidence.

Alternative execution choices. Table-free exact scoring is 30–52% slower (Table 3). Removing tables replaces reusable query-specific distances with arithmetic repeated for each candidate, while table construction occupies only 0.20–2.30% of measured batch time. Avoiding that small setup cost can therefore increase work repeated throughout the scan. Changes to probe traversal likewise alter throughput without changing logical code volume. These results show that repeated execution work matters alongside storage, but they do not identify instruction issue, cache behaviour, register pressure, or occupancy as the dominant hardware bottleneck.

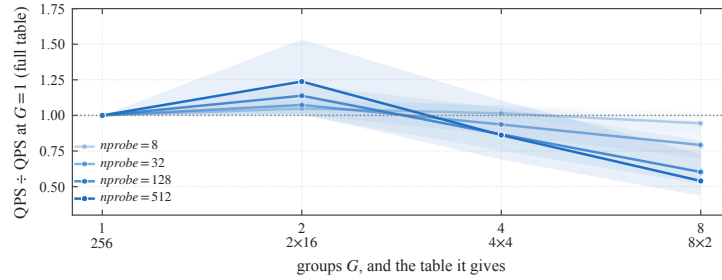


Fig. 3. Throughput against the full table, six datasets at four probe depths; lines are medians over datasets and bands their range. Factorising to $G=2$ gains, factorising further gives it back, and both effects grow with probe depth. 4×4 and 8×2 store the same 16 entries and require different numbers of lookups.

6.5 RQ4: Does the CPU refinement budget transfer to the GPU?

On OpenAI-3072, define the tuning gain as

$$g = \frac{\text{QPS}_{\text{tuned}}}{\text{QPS}_{100}} - 1,$$

where the tuned point is the fastest measured budget with at most 0.003 recall loss relative to each implementation’s own fixed- $\alpha = 100$ result. Across the shared probe depths 128–1024, $g_{\text{GPU}}/g_{\text{CPU}}$ is 2.5–3.0. Because reducing the survivor set removes residual work while leaving routing and primary screening intact, the benefit of changing α depends on the relative stage costs. The larger relative tuning gain in the measured GPU implementation therefore motivates retuning the inherited CPU budget rather than carrying it over unchanged.

The comparison is a sensitivity result, not an identical-index hardware speedup. The CPU uses 32 threads and separately trained codebooks, with 4,096 lists versus 8,192 on the OpenAI GPU frontiers. Probes below 128 are excluded because thread start-up dominates, most CPU configurations are timed once, and BGE and Stella have no CPU measurement. These differences limit attribution to hardware alone.

6.6 RQ5: Construction and memory costs

In Figure 4, the fastest compared alternative per dataset takes $3.5\text{--}8.5\times$ JHQ’s estimated build time. Training contributes 71% of JHQ’s estimate on Vogue but 27% on Stella: training dominates the former, while encoding dominates the latter. Both costs are paid before query-time savings accumulate, making construction relevant to workloads that rebuild frequently. Training budgets differ, including a reduced Stella IVF-PQ sample, so the comparison reflects the measured configurations rather than a controlled difference between quantisers.

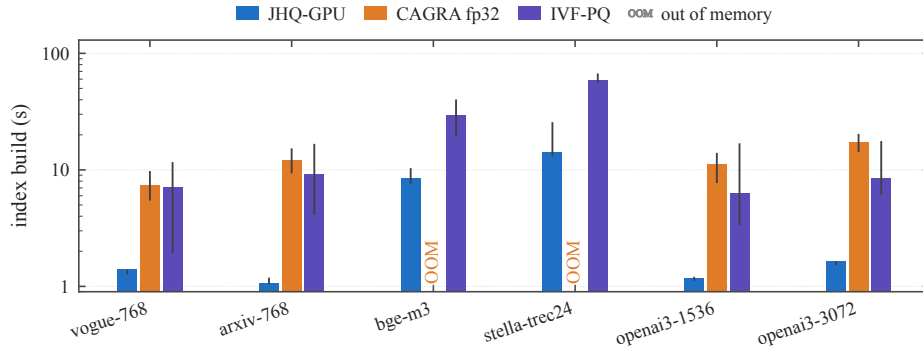


Fig. 4. Index build time, log scale. Bars are medians over clean configurations and repeats, whiskers their range, not a confidence interval. JHQ’s bar is one cold training plus its median encode.

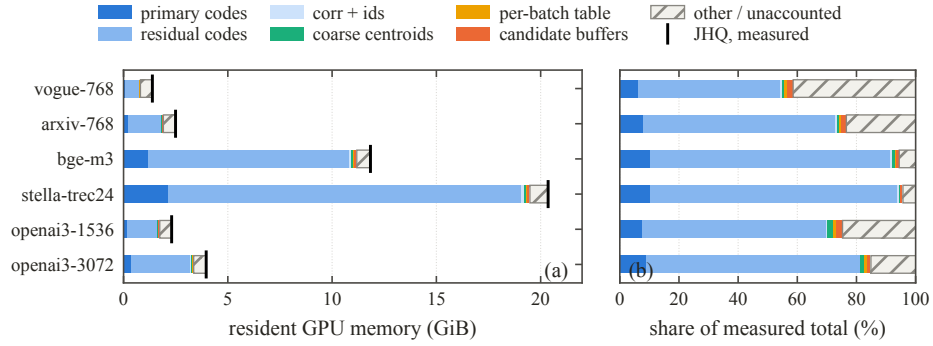


Fig. 5. Resident memory by component: (a) absolute, markers at measured totals; (b) the same stack as a share, where the five smaller sets are readable. Hatched is the unattributed remainder.

Residual codes remain the largest memory component (Figure 5): 48% of 1.4 GiB on Vogue and 83% of 20.4 GiB on Stella. Each vector retains eight residual bits per dimension alongside one primary bit. A smaller survivor budget reduces how many residual codes are read and scored per query but does not change their resident allocation; table factorisation reduces query-specific table state rather than residual storage. JHQ therefore trades resident residual capacity for selective query-time compute, occupying a different memory–accuracy–throughput operating point rather than being uniformly cheaper. The 4–41% unattributed memory remainder is reported separately.

7 Conclusion

JHQ-GPU combines exact Cartesian table factorization with workload-level refinement calibration. Two-group tables lead in 23/24 configurations, while equal-footprint alternatives confirm that lookup work also matters. Calibration yields $1.05\text{--}2.77\times$ fixed-budget throughput in 10/12 held-out configurations; arxiv instead gains mean recall at lower throughput. Output agreement does not guarantee quality: at $S = 128$, up to 40/64 draws lose more than 10^{-3} recall. Evidence is limited to one GPU, $k = 10$, and stationary query pools; other devices, k , and workload drift remain untested.

Acknowledgments. A large language model assistant was used throughout this work: to write and run the measurement scripts, to derive the figures and tables from the logs they produce, and to draft and revise the text. The authors designed the study, chose what to measure, and are responsible for every claim in the paper. JHQ itself is prior work and is not a contribution of this paper or of the assistant.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. André, F., Kermarrec, A.M., Scouarnec, N.L.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.* **9**(4), 288–299 (2015). <https://doi.org/10.14778/2856318.2856324>
2. André, F., Kermarrec, A.M., Scouarnec, N.L.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.* **43**(5), 1666–1677 (2021). <https://doi.org/10.1109/TPAMI.2019.2952606>
3. Assi, T.: Vogue933k embeddings. *Hugging Face Datasets* (2023), <https://huggingface.co/datasets/tonyassi/vogue933k-embeddings>
4. Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., Jégou, H.: The Faiss library. *arXiv:2401.08281* (2024)
5. Gao, J., Long, C.: Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data* **2**(3) (May 2024). <https://doi.org/10.1145/3654970>, <https://doi.org/10.1145/3654970>
6. Han, J., Zhang, M., Trajcevski, G.: JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.* **19**(7), 1530–1543 (2026). <https://doi.org/10.14778/3801059.3801067>
7. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**(1), 117–128 (2011). <https://doi.org/10.1109/TPAMI.2010.57>
8. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *arXiv:1702.08734* (2017)
9. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: *SIGMOD*. pp. 2539–2554 (2020). <https://doi.org/10.1145/3318464.3380600>
10. NVIDIA RAPIDS: cuVS: GPU-accelerated vector search and clustering. Software, version 26.08.01 (2026), <https://github.com/rapidsai/cuvs>
11. Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., Wang, Y.: CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In: *IEEE ICDE* (2024). <https://doi.org/10.1109/ICDE60146.2024.00323>
12. Qdrant: DBpedia entities: OpenAI text embedding 3 large, 1536 dimensions, 1m. *Hugging Face Datasets* (2024), <https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M>
13. Qdrant: DBpedia entities: OpenAI text embedding 3 large, 3072 dimensions, 1m. *Hugging Face Datasets* (2024), <https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M>
14. Shi, J., Gao, J., Xia, J., Fehér, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.* **19**(11), 3454–3467 (2026). <https://doi.org/10.14778/3836663.3836701>
15. The Alexandria Index: arXiv abstracts dataset. *Hugging Face Datasets* (2023), https://huggingface.co/datasets/macrocosm/arxiv_abstracts
16. The Information Engineering Lab: Stella TREC24 BioGen embedding. *Hugging Face Datasets* (2024), https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding_corpus_split
17. Upstash: Wikipedia 2024-06 BGE-M3. *Hugging Face Datasets* (2024), https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3_italian_configuration